

DameLibraries: Using Python Libraries from Tests

David Arroyo Menéndez (davidam@gmail.com)

Copyright © 2021 David Arroyo Menéndez

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the Appendix A entitled "License".

Table of Contents

1	Introduction	1
2	Installation	3
3	DameBasics	4
4	DameFormats	5
5	DameScraping	6
6	DameNumpy	7
7	DamePandas	8
8	DameScipy	9
9	DameScikit	10
10	DameNLTK	12
10.1	Sentiment Analysis	12
10.2	Detect Gender	13
10.3	Sentence Similarity	15
10.4	Stopwords	16
10.5	Manage words	16
10.6	Corpus	17
10.7	Gramatical Trees	17
11	DameWebutils: Checking images and urls ..	18
12	DameDB	20
12.1	SQLite	20
12.2	MySQL	20
12.3	Mongo	20
	Appendix A License	21
	Index	29

1 Introduction

Python is a winner in the trend market of programming languages in this moment. Although the syntax is so comfortable to the programmers, the clue is in the modern libraries for system administrators, scientifics and with low importance for web programmers.

You can see the Tiobe Index:

Jan 2021	Jan 2020	Change	Programming Language	Ratings	Change
1	2	▲	C	17.38%	+1.61%
2	1	▼	Java	11.96%	-4.93%
3	3		Python	11.72%	+2.01%
4	4		C++	7.56%	+1.99%
5	5		C#	3.95%	-1.40%
6	6		Visual Basic	3.84%	-1.44%
7	7		JavaScript	2.20%	-0.25%
8	8		PHP	1.99%	-0.41%
9	18	▲▲	R	1.90%	+1.10%
10	23	▲▲	Groovy	1.84%	+1.23%
11	15	▲▲	Assembly language	1.64%	+0.76%
12	10	▼	SQL	1.61%	+0.10%
13	9	▼▼	Swift	1.43%	-0.36%
14	14		Go	1.41%	+0.51%

You can check that Python is growing fast and reaching the most important positions in the market. The third position in January of 2021.

You can use or don't use a test system such as nose or pytest, but all people need test the code in the programming. After to write software you must introduce inputs in your code and verify if the output is ok.

The advantage using a test system is generate automation in the process about inputs, code being tested and outputs. Each test is evaluated with asserts, that's comparisons between the result that you deserve and the result obtained. For instance,

```
def test_models_lda(self):
    X = np.array([[-1, -1], [-2, -1], [-3, -2],
                  [1, 1], [2, 1], [3, 2]])
    y = np.array([1, 1, 1, 2, 2, 2])
    clf = LinearDiscriminantAnalysis()
    clf.fit(X, y)
    self.assertEqual(clf.predict([[-1, -1]]), 1)
```

So, assertEquals is the comparison and the rest is the code to be verified.

To become a professional in Python you must learn a good bunch of libraries. So you can practice the libraries reading tutorials, python notebooks, or you could write tests about this libraries and perhaps to share these tests in some software repository.

If you save all test code about your python libraries in one or various software repositories you can test all code with a command. The python libraries can suffer chances, but you have the control about the use that you do about your libraries.

The goal of this book is to teach Python Libraries from the tests.

2 Installation

You can create a python virtual environment and to install damelibraries with python package:

```
$ python3 -m venv /tmp/venv
$ cd /tmp/venv
$ source bin/activate
$ pip install --upgrade pip
$ pip3 install damelibraries
$ cd lib/python3.5/site-packages/damelibraries
```

Now you can execute tests to check different libraries:

```
$ cd /tmp/venv/lib/python3.8/site-packages/damealgorithms
$ nosetests3 tests
```

```
$ cd /tmp/venv/lib/python3.8/site-packages/dameformats
$ nosetests3 tests
```

```
$ cd /tmp/venv/lib/python3.8/site-packages/damenltk
$ nosetests3 tests
```

```
$ cd /tmp/venv/lib/python3.8/site-packages/damenumpy
$ nosetests3 tests
```

```
$ cd /tmp/venv/lib/python3.8/site-packages/damepandas
$ nosetests3 tests
```

```
$ cd /tmp/venv/lib/python3.8/site-packages/damescikit
$ nosetests3 tests
```

```
$ cd /tmp/venv/lib/python3.8/site-packages/damescipy
$ nosetests3 tests
```

3 DameBasics

Before, to start to learn python libraries you can remember python sintaxis about control structures such as if, while, for, ... and datastructures such as lists, dictionaries, tuples, sets, dates, ...

In DameBasics you can find some source snippets about it and execute it with:

```
$ cd /tmp/venv/lib/python3.8/site-packages/damebasics
$ pytest-3 tests
```

or

```
$ cd /tmp/venv/lib/python3.8/site-packages/damebasics
$ nosetests3 tests
```

4 DameFormats

Json, csv, xml are very important format files. This chapter is about how to use python with these formats presenting some snippets:

```
$ cd /tmp/venv/lib/python3.8/site-packages/dameformats
$ nosetests3 tests/test_damecsv.py
$ nosetests3 tests/test_damejson.py
$ nosetests3 tests/test_damexml.py
```


5 DameScraping

There are several libraries for to do scraping. The most standard and important library is request. I suggest you to use this library if you have doubts about what library to choose.

There are another libraries with modern features such as MechanicalSoup.

Another idea is libraries such as newspaper where you can integrate NLTK in a specific domain (newspaper).

Now you can check the source of the libraries.

```
$ cd /tmp/venv/lib/python3.8/site-packages/damescraping
$ nosetests3 test/test_requests.py
$ nosetests3 test/test_mechanicalsoup.py
$ nosetests3 test/test_newspaper.py
```

6 DameNumpy

Numpy is a library about algebra used as dependency in a lot of python programs. We have classified the tests in basics, datatypes and maths. Basics is related to the most common uses about the library (sum vectors, transpose matrix, generate eye matrix, ...). Datatypes is about specific uses about the numpy datatypes that perhaps you need practice more. And maths is about maths operations. You can execute the test with:

```
$ cd /tmp/venv/lib/python3.8/site-packages/damenumpy
$ nosetests3 tests/test_basics.py
$ nosetests3 tests/test_datatypes.py
$ nosetests3 tests/test_maths.py
```

7 DamePandas

Pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.

```
$ cd /tmp/venv/lib/python3.8/site-packages/damepandas
$ nosetests3 test/test_create.py
$ nosetests3 test/test_json.py
$ nosetests3 test/test_operators.py
$ nosetests3 test/test_utils.py
```

In test_create.py is being created tests for create datastructures. In test_json.py we practice some json utils in Pandas. In test_operators.py and test_utils.py there are more functions related to Pandas such as read_csv, concat, shape, ...

8 DameScipy

Scipy is a popular library focused on maths for science and computing released with BSD license. You can check the tests now:

```
$ cd /tmp/venv/lib/python3.8/site-packages/damescipy
$ nosetest3 tests
```

In `test_basics.py` you can learn useful maths such as to integrate, to interpolate, the distance between vectors, the polynomial roots, ...

In null hypothesis significance testing, the p-value is the probability of obtaining test results at least as extreme as the results actually observed, under the assumption that the null hypothesis is correct. In `test_pvalues.py` you can learn how to use it in python.

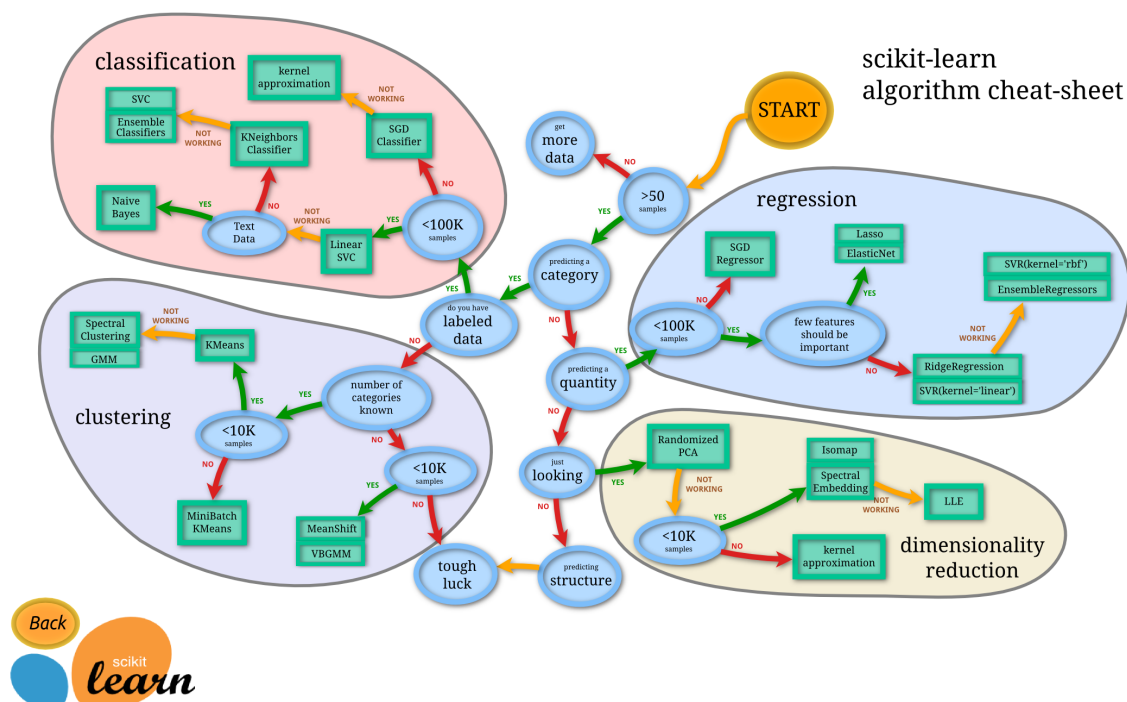
You can learn to use statistical distributions with `test_stats.py`.

9 DameScikit

Scikit is a popular library about Machine Learning (ML) released with BSD. In this chapter we are going to learn to apply algorithms to the different tasks in ML.

- Classification
- Clustering
- Clustering
- Regression
- Dimensionality Reduction

Take a look to the next diagram:



Classification and Clustering is about to learn to decide elements in groups (to classify). In Classification we have a training set, so we are doing supervised learning. In clustering we have not a training set, so we are doing unsupervised learning.

Dimensionality reduction is about to determine the most important factors or features to make the machine learning.

Regression is about to decide a quantity from a set of training examples.

In DameScikit we have a lot of examples about to apply machine learning models on a simple way:

```
$ nosetests3 tests/test_models.py
```

Take a look to a single test. Each array in the vector X (input) has an element in the vector Y (output). So $[-1, -1]$ has the representation 1 and $[2, 1]$ has the representation 2.

Later, we create an object `LinearDiscriminantAnalysis` (the model). Next, the model is fitted with inputs and outputs (train the model). Finally, we can predict inputs.

```
X = np.array([[-1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1], [3, 2]])
y = np.array([1, 1, 1, 2, 2, 2])
clf = LinearDiscriminantAnalysis()
clf.fit(X, y)
self.assertEqual(clf.predict([[-1, -1]]), 1)
```

That's the most important, but Scikit is a long library with a lot of interesting features. You can check more tests:

```
$ nosetests3 tests
```

10 DameNLTK

NLTK is an acronym about Natural Language ToolKit.

A summary of tasks about Natural Language Processing could be:

- Sentiment Analysis
- Detect Gender
- Sentence Similarity
- Text Summary
- Classify Documents
- Manage Words
- Gramatical Trees
- Extract Keywords
- Disambiguation

You could be in the path of nltk so:

```
$ cd /tmp/venv/  
$ source bin/activate  
(venv)$ pip3 install damenltk  
$ cd /tmp/venv/lib/python3.8/site-packages/damenltk  
$ nosetest3 test
```

10.1 Sentiment Analysis

Sentiment analysis is applied in a popular way in Twitter with python libraries such as tweepy. Finally, it is a classification task about words with a label (the sentiment). That could be implemented with nltk. Take a look to the test:

```
import unittest
from nltk.classify import SklearnClassifier
class TddInPythonExample(unittest.TestCase):

    def test_sentiment_variable_create_returns_correct_result(self):
        pos_tweets = [('I love this car', 'positive'),
                       ('This view is amazing', 'positive'),
                       ('I feel great this morning', 'positive'),
                       ('I am so excited about the concert', 'positive'),
                       ('He is my best friend', 'positive'),
                       ('This movie was great', 'positive'),
                       ('This movie was not pathetic', 'positive')]

        neg_tweets = [('I do not like this car', 'negative'),
                       ('This view is horrible', 'negative'),
                       ('I feel tired this morning', 'negative'),
                       ('I am not looking forward to the concert',
                        'negative'),
                       ('He is my enemy', 'negative'),
```

```

('This is a pathetic movie', 'negative']]

tweets_with_sentiment = []
for (tweet, sentiment) in pos_tweets + neg_tweets:
    filtered_tweet_words = [word.lower()
                             for word in tweet.split()
                             if len(word) >= 3]
    tweets_with_sentiment.append((filtered_tweet_words,
                                   sentiment))

self.assertEqual([(['love', 'this', 'car'], 'positive'),
                   ([ 'this', 'view', 'amazing'], 'positive'),
                   ([ 'feel', 'great', 'this', 'morning'],
                    'positive'),
                   ([ 'excited', 'about', 'the', 'concert'],
                    'positive'),
                   ([ 'best', 'friend'], 'positive'),
                   ([ 'this', 'movie', 'was', 'great'],
                    'positive'),
                   ([ 'this', 'movie', 'was', 'not', 'pathetic'],
                    'positive'),
                   ([ 'not', 'like', 'this', 'car'], 'negative'),
                   ([ 'this', 'view', 'horrible'], 'negative'),
                   ([ 'feel', 'tired', 'this', 'morning'],
                    'negative'),
                   ([ 'not', 'looking', 'forward', 'the', 'concert'],
                    'negative'),
                   ([ 'enemy'], 'negative'),
                   ([ 'this', 'pathetic', 'movie'], 'negative')],
                  tweets_with_sentiment)

```

You can execute this test with:

```
$ nosetest3 test/test_sentiment.py
```

10.2 Detect Gender

The gender detection from the name is a NLP task. So the input is one or more word (name and perhaps surname) and the output is a gender male, female, ...

NLTK brings a good list of names in the `nltk.corpus`. So you can classify names with two ways:

- Querying single files
- Supervised classifier

The python method using single files would be:

```

def gender_name(self, name):
    gender = 'undefined'
    if name in names.words('male.txt'):

```



```

        gender = 'male'
    if name in names.words('female.txt'):
        if (gender == 'male'):
            gender = 'both'
        else:
            gender = 'female'
    return gender

```

And the python test would be:

```

def test_gender_name_method_returns_correct_result(self):
    dn = DameNLTK()
    self.assertTrue(
        dn.gender_name("Neo"),
        "male")
    self.assertTrue(
        dn.gender_name("Trinity"),
        "female")
    self.assertTrue(
        dn.gender_name("Andrea"),
        "both")

```

The python methods using machine learnig would be:

```

def gender_features(self, word):
    return 'last_letter': word[-1]

def gender_classifier(self):
    labeled_names = (
        [(name, 'male') for name in names.words('male.txt')] +
        [(name, 'female') for name in names.words('female.txt')])

    featuresets = [(self.gender_features(n), gender)
                    for (n, gender) in labeled_names]
    train_set, test_set = featuresets[500:], featuresets[:500]
    classifier = nltk.NaiveBayesClassifier.train(train_set)
    return classifier

```

And the python tests would be:

```

def test_gender_classifier_method_returns_correct_result(self):
    dn = DameNLTK()
    classifier = dn.gender_classifier()
    self.assertTrue(classifier.classify(dn.gender_features("Neo")),
                    "male")
    self.assertTrue(classifier.classify(dn.gender_features("Trinity")),
                    "female")

```

You can use an advanced software to detect gender such as Damegender with many features:

```
$ pip3 install damegender[all]
```

You can learn to use the software with tests oriented to the user (bash files) or tests oriented to the developer (python tests).

```
$ python3 -m venv /tmp/damegender/
$ cd /tmp/damegender/
$ source bin/activate
$ pip3 install damegender[all]
$ ./testsbycommands.sh
maindavid test is ok
mainjesus test is ok
mainines test is ok
mainalex test is ok
mainandrea test is ok
mainjesusmaria test is ok
mainjosemaria test is ok
mainelenaluciahelena test is ok
mainjuliaus test is ok
mainjuliauk test is ok
mainjuliauy test is ok
[...]

$ nosetest3 test
[...]
```

Execute these tests could be long time, but you can execute a single file test, for example, you can execute `test/test_dame_utils.py`:

```
$ nosetest3 test/test_dame_utils.py
```

Or you can execute a single test in a file with:

```
$ nosetests3 test/test_dame_utils.py:TddInPythonExample.test_string2array
```

10.3 Sentence Similarity

Remembering to W H Gomaa and A A Fahmy in “A Survey of Text Similarity Approaches”:

Finding similarity between words is a fundamental part of text similarity which is then used as a primary stage for sentence, paragraph and document similarities. Words can be similar in two ways lexically and semantically. Words are similar lexically if they have a similar character sequence. Words are similar semantically if they have the same thing, are opposite of each other, used in the same way, used in the same context and one is a type of another.

In the python method `sentence_similarity` we apply similarity lexical. So, it's dividing the text in two vectors (`vector1` and `vector2`) and we apply the cosine distance:

```
def sentence_similarity(self, sent1, sent2, stopwords=None):
    if stopwords is None:
        stopwords = []

    sent1 = [w.lower() for w in sent1]
    sent2 = [w.lower() for w in sent2]
```

```

all_words = list(set(sent1 + sent2))

vector1 = [0] * len(all_words)
vector2 = [0] * len(all_words)

# build the vector for the first sentence
for w in sent1:
    if w in stopwords:
        continue
    vector1[all_words.index(w)] += 1

# build the vector for the second sentence
for w in sent2:
    if w in stopwords:
        continue
    vector2[all_words.index(w)] += 1

return 1 - cosine_distance(vector1, vector2)

```

We can learn to use this method checking the test:

```

def test_sentencesimilarity_method_returns_correct_result(self):
    dn = DameNLTK()
    self.assertTrue(dn.sentence_similarity(
        "This is a good sentence".split(),
        "This is a bad sentence".split()) > 0.6)

```

Remember execute the test to learn if the test is running:

```
$ nosetest3 test/test_sentence_similarity.py
```

10.4 Stopwords

For some search engines, these are some of the most common, short function words, such as the, is, at, which, and on. In this case, stop words can cause problems when searching for phrases that include them

You can find the test:

```
$ nosetest3 test/test_stopwords.py
```

10.5 Manage words

Managing words is about definitions, singulars, plurals, synonyms and antonyms

We can start building singulars and plurals from a word. You can read the test file:

```

import unittest
import nltk
from nltk.stem import *
from nltk.stem.snowball import SnowballStemmer

class TddInPythonExample(unittest.TestCase):

```

```
def test_stem_returns_correct_result(self):
    stemmer = PorterStemmer()
    self.assertEqual(stemmer.stem("vuelos"), "vuelo")

def test_snowball_stem_returns_correct_result(self):
    sb = SnowballStemmer("english").stem("generously")
    self.assertEqual(sb, "generous")
```

You can test it with:

```
$ nosetest3 test/test_stem.py
```

Another tool for NLP tasks is to retrieve synonyms and antonyms from words. You can execute the tests:

```
$ nosetest3 test/test_syn.py
```

Another way is using wordnet:

```
$ nosetests3 test/test_wordnet.py
```

Wordnet provides measures about semantic similarity between words, definitions about words, ...

10.6 Corpus

The corpus allows use data very useful in NLP (ex: gutemberg files, twitter examples, wordnet, names with gender, ...). You can check the source to learn how to use it:

```
$ nosetests3 test/test_corpus.py
$ nosetests3 test/test_gutenberg.py
```

10.7 Gramatical Trees

The gramatical trees are a tool in Natural Language Processing to draw the gramatical in a visual or computing way. We have created tests with gramatical trees:

```
$ nosetests3 test/test_semantic_tree.py
```

11 DameWebutils: Checking images and urls

You can use python testing in Django or Flask on a similar way than in another libraries, but there are some tasks as check broken urls or images that you can use in any web site. That's the subject of this chapter.

Now, you can install damewebutils easily in a python virtual environment:

```
davidam-Lenovo-B50-10:~/git/damewebutils$ mkdir /tmp/venv
davidam-Lenovo-B50-10:~/git/damewebutils$ python3 -m venv /tmp/venv/
davidam-Lenovo-B50-10:~/git/damewebutils$ cd /tmp/venv/
davidam-Lenovo-B50-10:/tmp/venv$ ls
bin  include  lib  lib64  pyvenv.cfg  share
davidam-Lenovo-B50-10:/tmp/venv$ source bin/activate
(venv) davidam-Lenovo-B50-10:/tmp/venv$ pip3 install damewebutils
Processing /home/davidam/.cache/pip/wheels/6a/92/21/f0210ab109dd30d573ec84c79c7a2cc556
0.1.2-py3-none-any.whl
Collecting markdown
  Downloading Markdown-3.3.4-py3-none-any.whl (97 kB)
    || 97 kB 565 kB/s
Collecting requests
  Using cached requests-2.25.1-py2.py3-none-any.whl (61 kB)
Collecting lxml
  Using cached lxml-4.6.2-cp38-cp38-manylinux1_x86_64.whl (5.4 MB)
Collecting cssselect
  Using cached cssselect-1.1.0-py2.py3-none-any.whl (16 kB)
Collecting idna<3,>=2.5
  Using cached idna-2.10-py2.py3-none-any.whl (58 kB)
Collecting certifi>=2017.4.17
  Using cached certifi-2020.12.5-py2.py3-none-any.whl (147 kB)
Collecting urllib3<1.27,>=1.21.1
  Using cached urllib3-1.26.3-py2.py3-none-any.whl (137 kB)
Collecting chardet<5,>=3.0.2
  Using cached chardet-4.0.0-py2.py3-none-any.whl (178 kB)
Installing collected packages: markdown, idna, certifi, urllib3, chardet, re-
quests, lxml, cssselect, damewebutils
Successfully installed certifi-2020.12.5 chardet-4.0.0 cssselect-1.1.0 damewebutils-
0.1.2 idna-2.10 lxml-4.6.2 markdown-3.3.4 requests-2.25.1 urllib3-1.26.3
```

Now you can execute dameimgs.py and dameurls.py giving some url as argument. For example:

```
(venv) davidam-Lenovo-B50-10:/tmp/venv$ dameimgs.py https://www.davidam.com
https://www.davidam.com
https://www.davidam.com/img/davidam-pypi.jpg
200
https://www.davidam.com/img/davidam-pypi.jpg
200
Imgs with possible troubles in []
```

In this example, we have found one image using the html markup on a right way.

With the urls is similar:

```
$ dameurls.py https://www.davidam.com
https://twitter.com/davidam9
https://scholar.google.es/citations?user=znKwqIMAAAAJ&hl=es&oi=ao
https://orcid.org/0000-0002-2986-5361
https://code.orgmode.org/bzg/org-mode/src/master/contrib/lisp/org-effectiveness.el
https://code.orgmode.org/bzg/org-mode/src/master/contrib/lisp/org-license.el
https://savannah.nongnu.org/projects/drupal-el
https://savannah.nongnu.org/projects/php-ext-el
http://drupal.org/project/orgmode
http://drupal.org/project/ocrad
https://github.com/davidam/damegender
[...]
Urls with troubles:
['https://github.com/davidam/damegender/blob/dev/manual/damegender.es.pdf']
```

12 DameDB

12.1 SQLite

SQLite is the most simple SQL database. You can create and check a sql datamodel executing the next tests:

```
$ nosetests3 tests/test_damedb_sqlite.py
```

12.2 MySQL

MySQL is the most used free sql database.

You need create a database model with:

```
$ mysql -u root < files/create.sql
```

Later you can check python operation with the database:

```
$ nosetests3 tests/test_damedb_mysql.py
```

12.3 Mongo

Mongo is a nosql database very used in the market.

The first step to use Mongo in Python is to install Mongo in a computer. You can read the manual to install mongo in different operating systems <https://docs.mongodb.com/manual/tutorial/>.

The second step is to install python libraries about mongo. For example, mongoengine.

```
python -m pip install mongoengine
```

Later you can make exercises about mongo in python. Run your tests!

```
$ nosetests3 tests/test_damedb_mongo.py
```

Appendix A License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.
<https://fsf.org/>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document *free* in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or non-commercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of “copyleft”, which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The “Document”, below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as “you”. You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A “Modified Version” of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A “Secondary Section” is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document’s overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The “Invariant Sections” are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released

under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The “Cover Texts” are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A “Transparent” copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not “Transparent” is called “Opaque”.

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The “Title Page” means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, “Title Page” means the text near the most prominent appearance of the work’s title, preceding the beginning of the body of the text.

The “publisher” means any person or entity that distributes copies of the Document to the public.

A section “Entitled XYZ” means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as “Acknowledgements”, “Dedications”, “Endorsements”, or “History”.) To “Preserve the Title” of such a section when you modify the Document means that it remains a section “Entitled XYZ” according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any,

- be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.
- B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.
 - C. State on the Title page the name of the publisher of the Modified Version, as the publisher.
 - D. Preserve all the copyright notices of the Document.
 - E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
 - F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
 - G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
 - H. Include an unaltered copy of this License.
 - I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
 - J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.
 - K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.
 - L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
 - M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.
 - N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.
 - O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their

titles to the list of Invariant Sections in the Modified Version’s license notice. These titles must be distinct from any other section titles.

You may add a section Entitled “Endorsements”, provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled “History” in the various original documents, forming one section Entitled “History”; likewise combine any sections Entitled “Acknowledgements”, and any sections Entitled “Dedications”. You must delete all sections Entitled “Endorsements.”

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an “aggregate” if the copyright resulting from the compilation is not used to limit the legal rights of the compilation’s users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document’s Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled “Acknowledgements”, “Dedications”, or “History”, the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <https://www.gnu.org/licenses/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License “or any later version” applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies that a proxy can decide which future versions of this License can be used, that proxy’s public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

11. RELICENSING

“Massive Multiauthor Collaboration Site” (or “MMC Site”) means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A “Massive Multiauthor Collaboration” (or “MMC”) contained in the site means any set of copyrightable works thus published on the MMC site.

“CC-BY-SA” means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

“Incorporate” means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is “eligible for relicensing” if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.

ADDENDUM: How to use this License for your documents

To use this License in a document you have written, include a copy of the License in the document and put the following copyright and license notices just after the title page:

```
Copyright (C)  year  your name.
Permission is granted to copy, distribute and/or modify this document
under the terms of the GNU Free Documentation License, Version 1.3
or any later version published by the Free Software Foundation;
with no Invariant Sections, no Front-Cover Texts, and no Back-Cover
Texts. A copy of the license is included in the section entitled ‘‘GNU
Free Documentation License’’.
```

If you have Invariant Sections, Front-Cover Texts and Back-Cover Texts, replace the “with...Texts.” line with this:

```
with the Invariant Sections being list their titles, with
the Front-Cover Texts being list, and with the Back-Cover Texts
being list.
```

If you have Invariant Sections without Cover Texts, or some other combination of the three, merge those two alternatives to suit the situation.

If your document contains nontrivial examples of program code, we recommend releasing these examples in parallel under your choice of free software license, such as the GNU General Public License, to permit their use in free software.

Index

C

Checking images	18
Checking urls	18

D

DameBasics	4
DameDB	20
DameFormats	5
DameNLTK	12
DameNumpy	7
DamePandas	8
DameScikit	10
DameScipy	9
DameScraping	6

DameWebutils	18
Databases	20

I

Installation	3
Introduction	1

L

License	21
---------------	----

P

Python Virtual Environment	3
----------------------------------	---